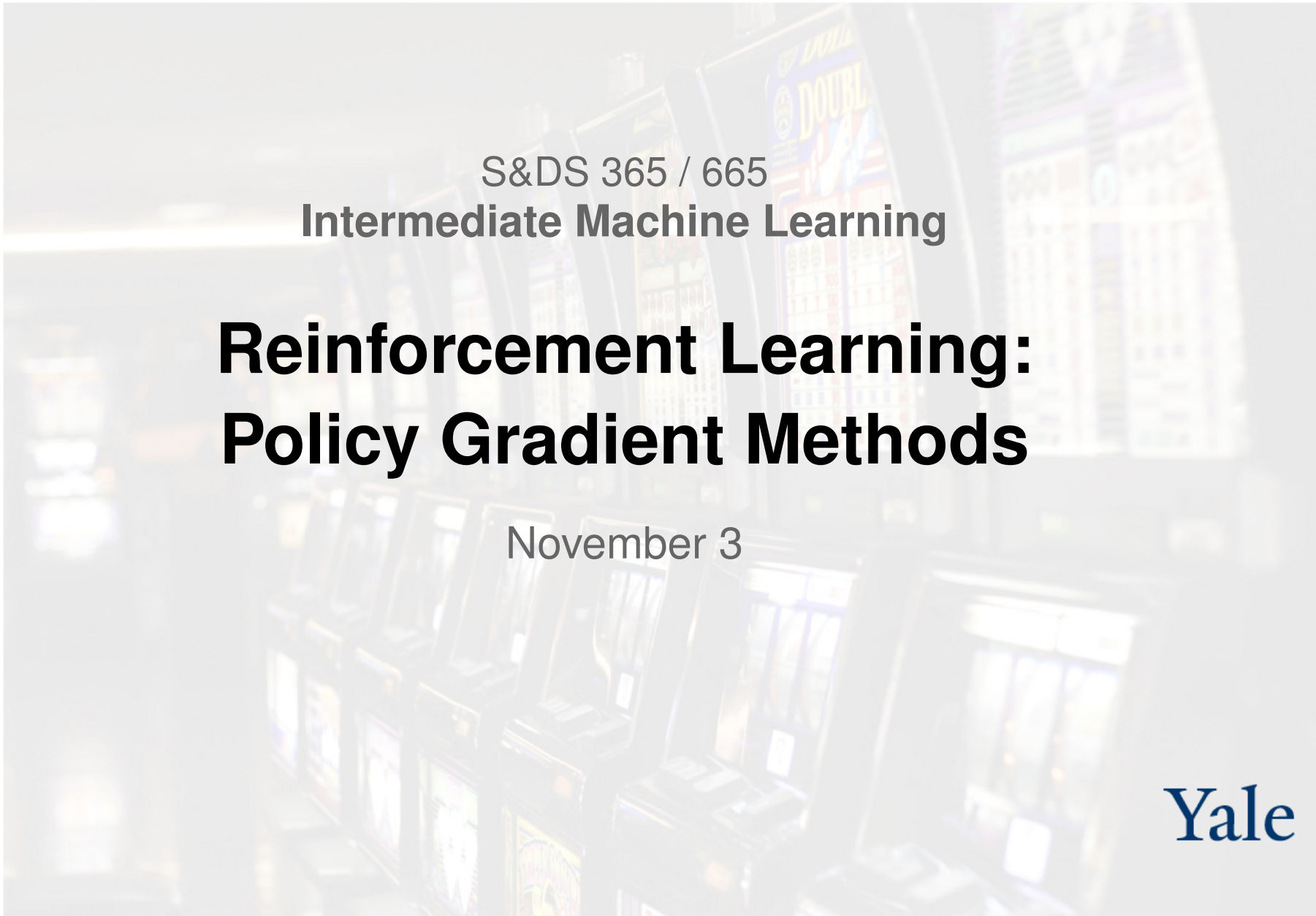


S&DS 365 / 665

Intermediate Machine Learning

Reinforcement Learning: Policy Gradient Methods

November 3

Yale

Version of 2025/11/03 at 10:28

Reminders

- Quiz 4 on Wednesday, November 5
  - Graphs and conditional independence
  - Laplacians and GNNs
  - Q-learning
  - Midterm
- Assignment 4 is out; due Thursday, November 20

Outline

- Deep Q-Learning: Recap
- Automatic differentiation
- Minimal DQN example
- Policy gradients
- Using a baseline average

Main reference: “Reinforcement Learning: An Introduction” (Second Edition), Richard S. Sutton and Andrew G. Barto

Principle: Bellman equation

Value function optimality

$$v_*(s) = \max_a \mathbb{E}\Big[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a\Big]$$

Principle: Bellman equation

Q-function optimality

$$Q_*(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q_*(S_{t+1}, a') \mid S_t = s, A_t = a\Big]$$

Algorithm from this principle

Q-Learning

Parameters: step size α, exploration probability ε, discount factor γ
Initialize $Q(s, a)$ arbitrarily, except $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:
    Initialize state s
    Loop for each step of episode:
        Choose action a using Q with ε-greedy policy
        Take action a; observe reward r and new state s'
        $Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right)$
        $s \leftarrow s'$
    Until s is terminal

Comment on Q-learning

- Q-learning is an example of *temporal difference (TD) learning*
- It is an "off-policy" approach that is practical if the space of states and actions is small
- Value iteration is analogous approach for learning value function

Deep reinforcement learning: Motivation

- Direct implementation of Q-learning only possible for small state and action spaces
- For large state spaces we need to map states to “features”
- Deep RL uses a multilayer neural network to learn these features and the Q-function

Strategy

Bellman equation (deterministic environment):

$$Q(s, a; \theta) = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta)$$

- Let y_t be one step of "play":

$$y_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta_{\text{old}})$$

- Adjust the parameters θ to make the squared error small (SGD):

$$(y_t - Q(s, a; \theta))^2$$

- Collect many such steps before updating gradient (replay buffer)

Strategy

Adjust the parameters θ to make the squared error small

$$(y_t - Q(s, a; \theta))^2$$

How? Carry out SGD

$$\theta \longleftarrow \theta + \eta \left(y_t - Q(s, a; \theta)\right) \nabla_\theta Q(s, a; \theta)$$

using backpropagation

Replay buffer

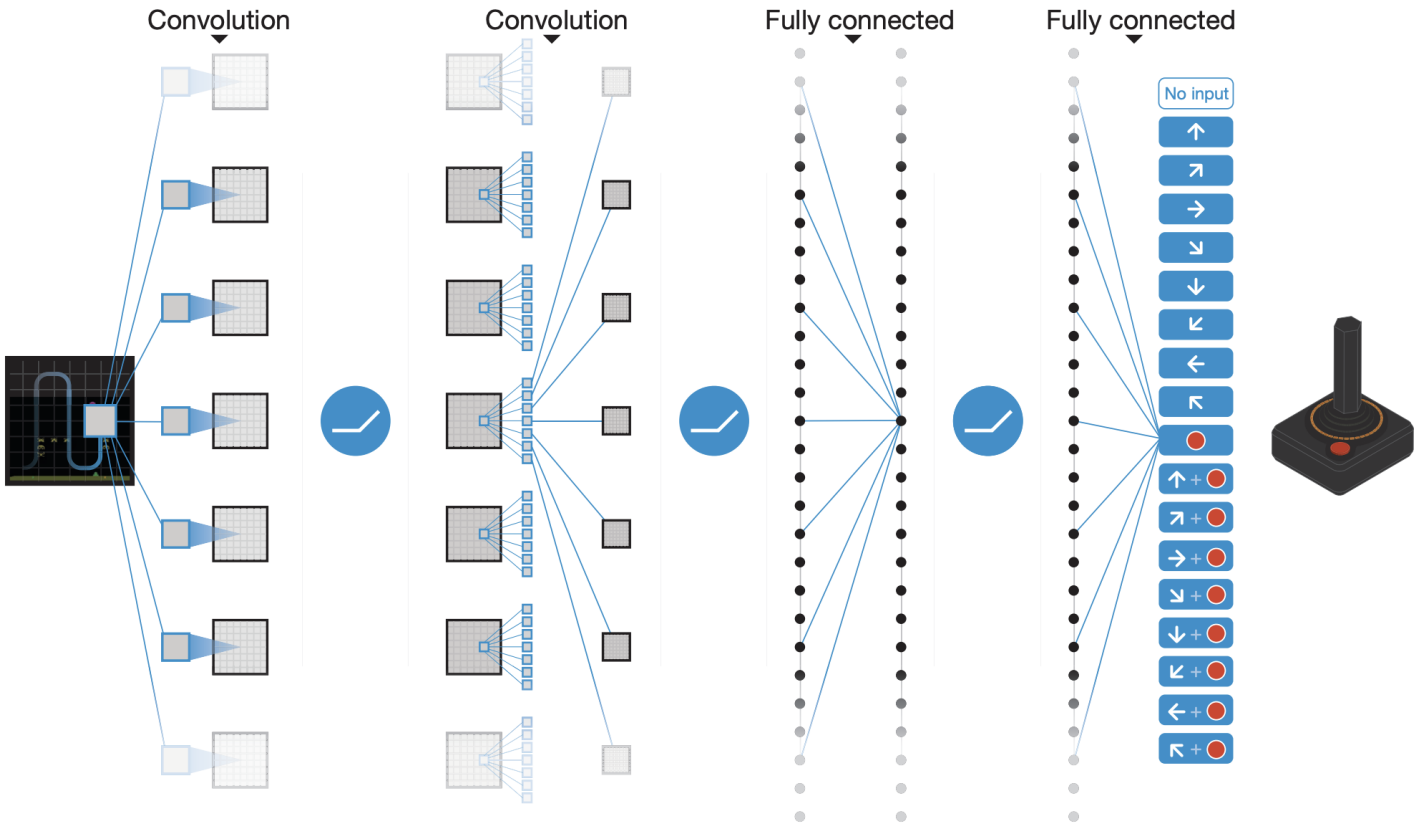
- *Q*-learning carried out using minibatches from a replay buffer of sequences that are “remembered and replayed”

Replay buffer

Role of replay buffer (inspired by neuroscience, “dreaming”)

- Prevent “forgetting” how to play early parts of a game
- Remove correlations between nearby state transitions
- Prevent cycling behavior, due to target changing

Second generation DQN



https://storage.googleapis.com/deepmind-data/assets/papers/DeepMindNature14236Paper.pdf

When does learning take place?

Recall that Bellman equation is a constraint on Q as an expectation.

Learning takes place when expectations are violated. The receipt of the reward itself does not cause changes.

Neuroscience connection

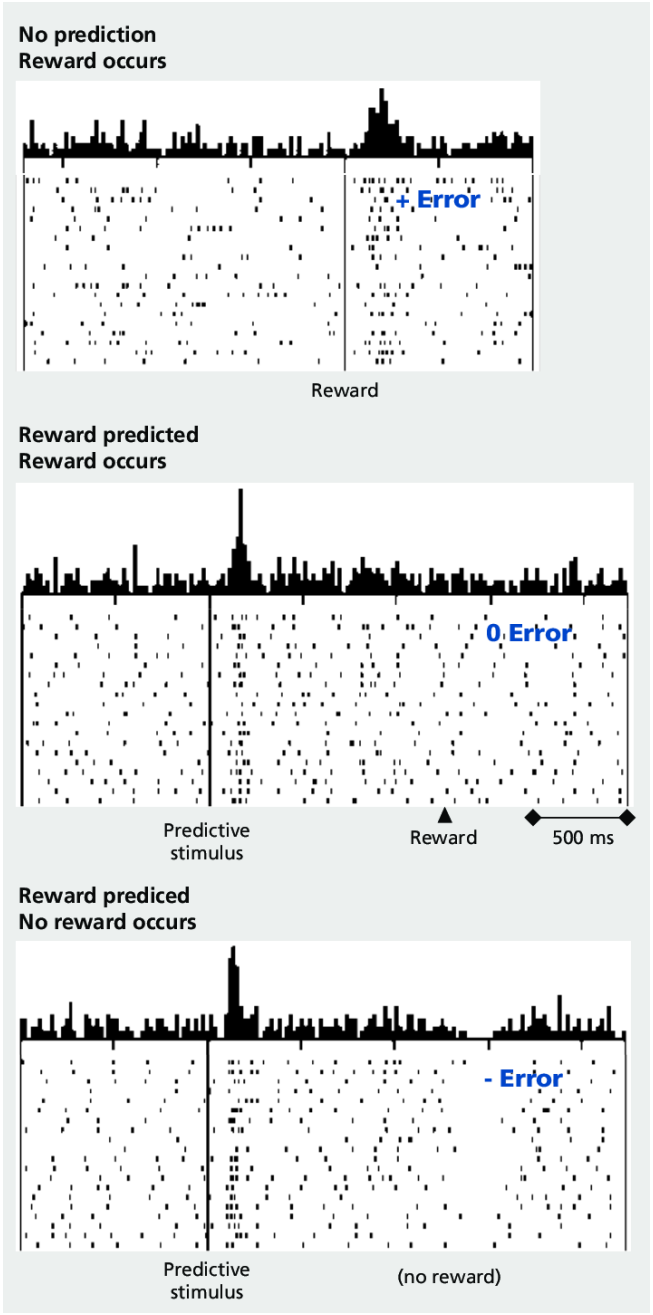
A Neural Substrate of Prediction and Reward

Wolfram Schultz, Peter Dayan, P. Read Montague*

The capacity to predict future events permits a creature to detect, model, and manipulate the causal structure of its interactions with its environment. Behavioral experiments suggest that learning is driven by changes in the expectations about future salient events such as rewards and punishments. Physiological work has recently complemented these studies by identifying dopaminergic neurons in the primate whose fluctuating output apparently signals changes or errors in the predictions of future salient and rewarding events. Taken together, these findings can be understood through quantitative theories of adaptive optimizing control.

Science 1997

Neuroscience connection



Automatic differentiation

- For supervised problems, loss function is $L(\widehat{Y}, Y)$
- Can program this directly
- RL loss functions are built up dynamically as agent makes decisions
- Automatic differentiation allows us to handle this

`https://www.tensorflow.org/guide/autodiff`

`https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html`

Automatic differentiation: Gradient collection

TensorFlow supports automatic differentiation by recording relevant operations executed inside the context of a “tape”
`tf.GradientTape`

It then uses the record to compute the numerical values of gradients using “reverse mode differentiation”

---

https://en.wikipedia.org/wiki/Automatic_differentiation

Automatic differentiation: Hello world!

In [1]:
```
import numpy as np
import tensorflow as tf
```

In [2]:
```
x = tf.Variable(3.0)

with tf.GradientTape() as tape:
   y = x**2

dy_dx = tape.gradient(y, x)
print(dy_dx.numpy())
```

```
6.0
```

https://www.tensorflow.org/guide/autodiff

Automatic differentiation: Hello world!

In [3]:
```
w = tf.Variable(tf.random.normal((3, 2)), name='w')
b = tf.Variable(tf.zeros(2, dtype=tf.float32), name='b')
x = [[1., 2., 3.]]

with tf.GradientTape() as tape:
    y = x @ w + b
    loss = tf.reduce_mean(y**2)

[dloss_dw, dloss_db] = tape.gradient(loss, [w, b])
print(dloss_dw.numpy(), "\n\n", dloss_db.numpy())
```

```
[[2.3997068  0.70033383]
 [4.7994137  1.4006677 ]
 [7.1991205  2.1010015 ]]

 [2.3997068  0.70033383]
```

https://www.tensorflow.org/guide/autodiff

Automatic differentiation: Parameter updates

Parameters are then updated as shown here:

In [4]:
```
opt = keras.optimizers.Adam(learning_rate=0.001)
_ = opt.apply_gradients(zip([dloss_dw, dloss_db], [w, b]))
```

We'll see examples shortly

https://www.tensorflow.org/guide/autodiff

Multi-armed bandits

Multi-armed bandits

- The rewards are independent and noisy
- Arm k has expected payoff μ_k with variance σ_k^2 on each pull
- Each time step, pull an arm and observe the resulting reward
- Played often enough, can estimate mean reward of each arm
- What is the best policy?
- Exploration-exploitation tradeoff
- "Contextual bandits" add covariate vector — but actions don't impact the environment

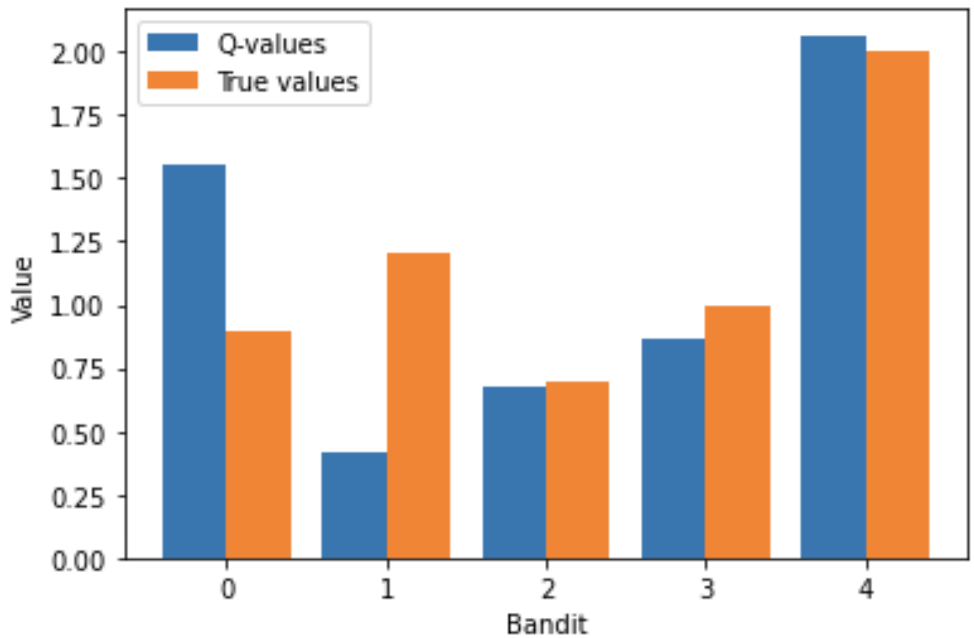
Multi-armed bandits

We'll treat this as an RL problem and hit it with a big hammer:
Deep Q-learning

Multi-armed bandits

```
======episode 10000 ======
Q-values ['1.556', '0.412', '0.675', '0.866', '2.065']
Deviation ['72.8%', '-65.7%', '-3.6%', '-13.4%', '3.3%']

<Figure size 864x504 with 0 Axes>
```

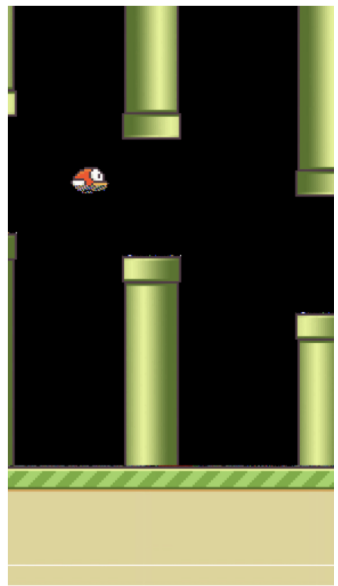


Let’s go to the notebook!

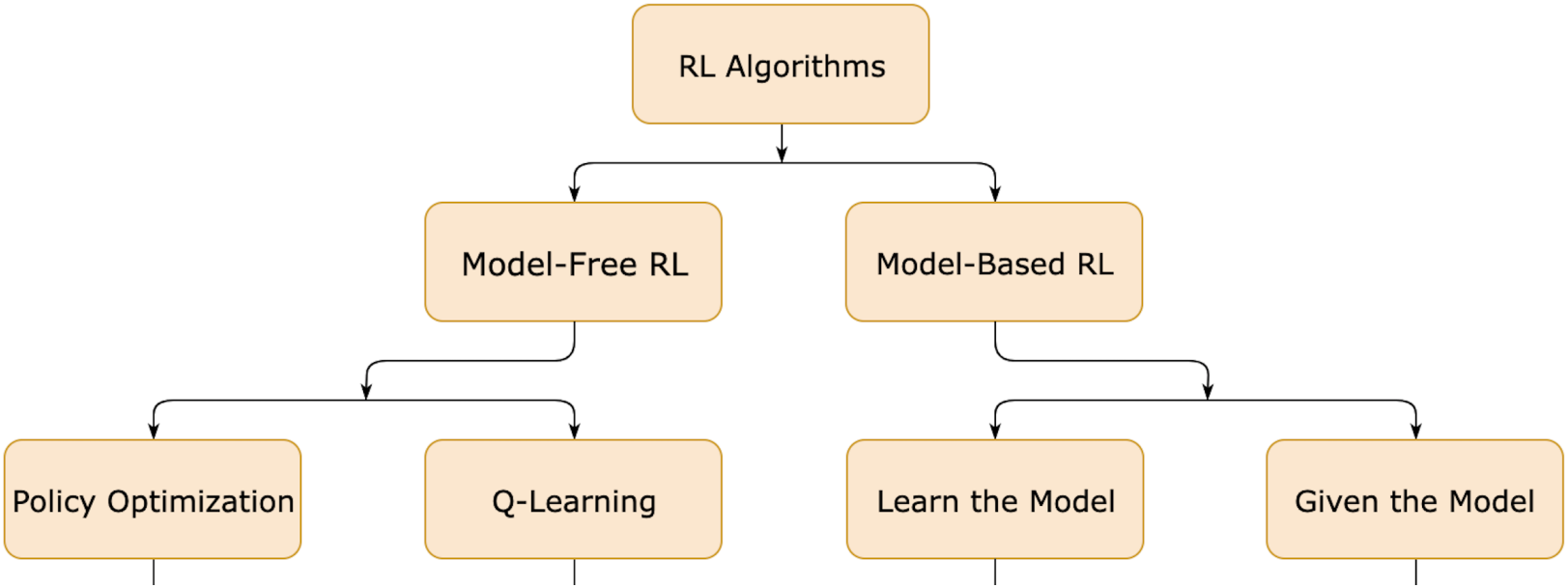
Assn 4: Flappy Bird

Problem 3: Deep Q-Learning for Flappy Bird (25 points)

In this problem, we will walk you through the implementation of deep Q learning to learn to play the Flappy Bird game.



Landscape of RL algorithms

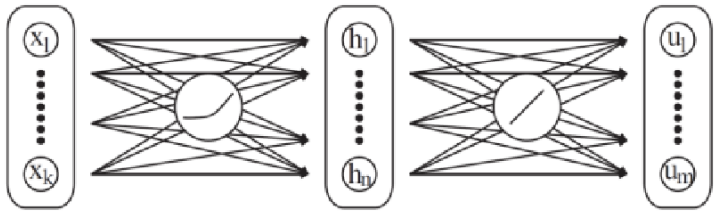


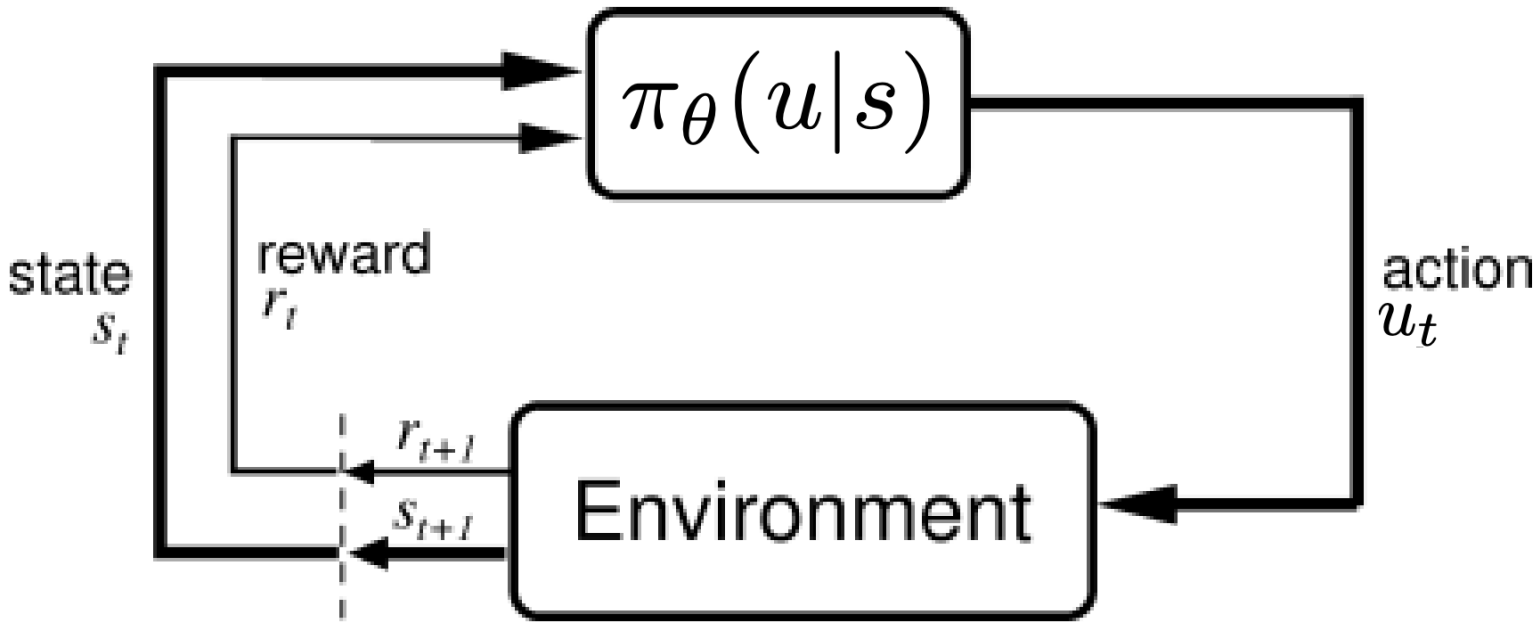
https://tinyurl.com/25vruwtp

Policy gradient methods

- policy $\pi_\theta(s)$ has parameters θ
- Perform gradient ascent over θ
- Well-suited to deep learning approaches
- Why use an on-policy method? May be possible to estimate a good policy without accurately estimating the value function or Q-function
- Can be more compute-efficient and stable than Q-learning

Policy gradient methods





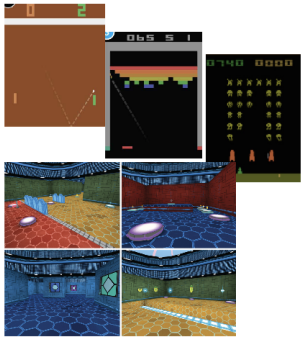
Successes of policy methods

Kohl and Stone, 2004

Ng et al, 2004

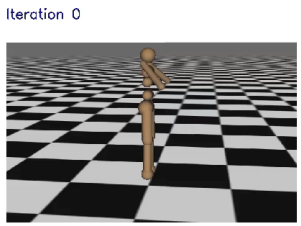
Tedrake et al, 2005

Kober and Peters, 2009

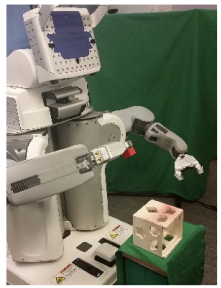


Mnih et al, 2015
(A3C)

Silver et al, 2014
(DPG)
Lillicrap et al, 2015
(DDPG)



Schulman et al,
2016 (TRPO + GAE)



Levine*, Finn*, et
al, 2016
(GPS)

Silver*, Huang*, et
al, 2016
(AlphaGo**)

Policy gradient methods: Loss function

We start with the loss function: Expected reward $\mathcal{J}(\theta) = \mathbb{E}(R)$

- Policy $\pi(s; \theta)$ has parameters θ features of states
- Perform gradient ascent on $\mathcal{J}(\theta)$
- Well-suited to deep learning approaches

We'll need to develop some mathematics to understand how policy methods can be efficiently applied...

Policy gradient methods: Loss function

Policy is probability distribution $\pi_\theta(a \mid s)$ over actions given state s.

The episode unfolds as a random sequence τ

$$\tau : (s_0, a_0) \to (s_1, r_1, a_1) \to (s_2, r_2, a_2) \to \cdots \to (s_T, r_T, a_T) \to s_{T+1}$$

where s_{T+1} is a terminal state. Receive reward $R(\tau)$, for example

$$R(\tau) = \sum_{t=1}^{T} r_t$$

Objective function $\mathcal{J}$ is expected reward

$$\mathcal{J}(\theta) = \mathbb{E}_\theta(R(\tau))$$

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\tau$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\tau$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

It follows that

$$\nabla_\theta \log p(\tau \mid \theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) = \sum_{t=0}^{T} \frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

Calculating the gradient

Now we use

$$
\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &= \nabla_\theta \, \mathbb{E}_\theta R(\tau) \\
&= \nabla_\theta \int R(\tau) \; p(\tau \,|\, \theta) \; d\tau \\
&= \int R(\tau) \, \nabla_\theta p(\tau \,|\, \theta) \; d\tau \\
&= \int R(\tau) \, \frac{\nabla_\theta p(\tau \,|\, \theta)}{p(\tau \,|\, \theta)} \; p(\tau \,|\, \theta) \; d\tau \\
&= \mathbb{E}_\theta \Big(R(\tau) \nabla_\theta \log p(\tau \,|\, \theta) \Big)
\end{aligned}
$$

Need regularity conditions to exchange derivative and integral, such as uniform continuity of the derivatives of the policy.

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \nabla_\theta \log p(\tau^{(i)} \mid \theta) \\
&= \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\
&\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}
\end{aligned}$$

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)})\nabla_\theta \log p(\tau^{(i)} \mid \theta) \\ &= \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\ &\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}\end{aligned}$$

$\tau^{(i)}$ is sometimes called a "rollout" of the policy

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)})\nabla_\theta \log p(\tau^{(i)} \mid \theta) \\ &= \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\ &\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}\end{aligned}$$

$\tau^{(i)}$ is sometimes called a "rollout" of the policy

The policy gradient algorithm is then

$$\theta \longleftarrow \theta + \eta \widehat{\nabla_\theta \mathcal{J}(\theta)}$$

Intuition

Gradient ascent of the expected reward with respect to policy:

- Increases probability of paths with large R
- Decreases probability of paths with small R

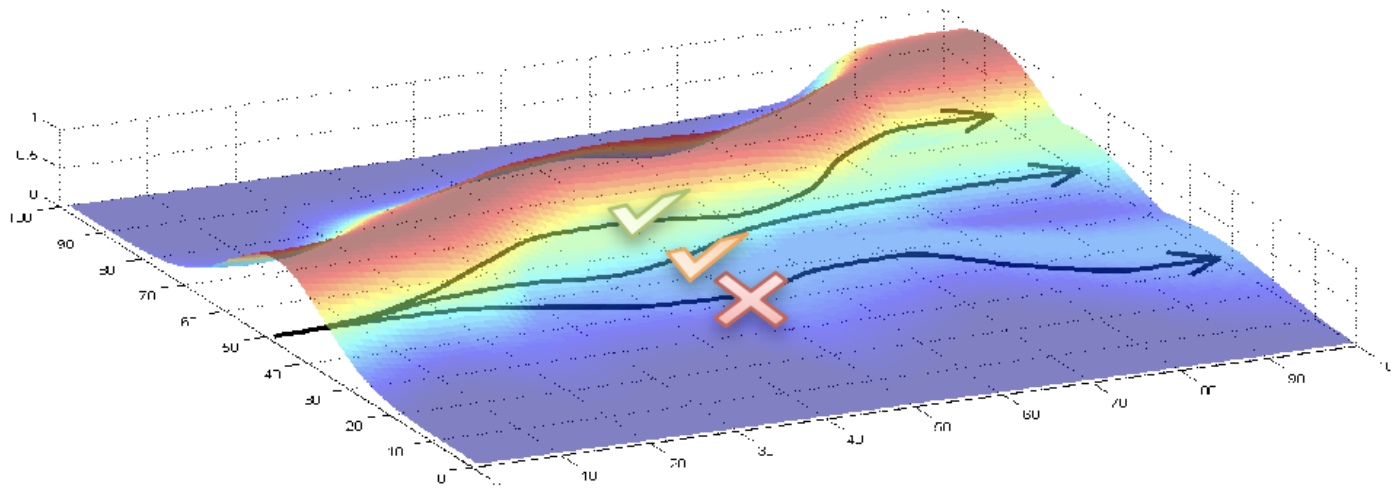
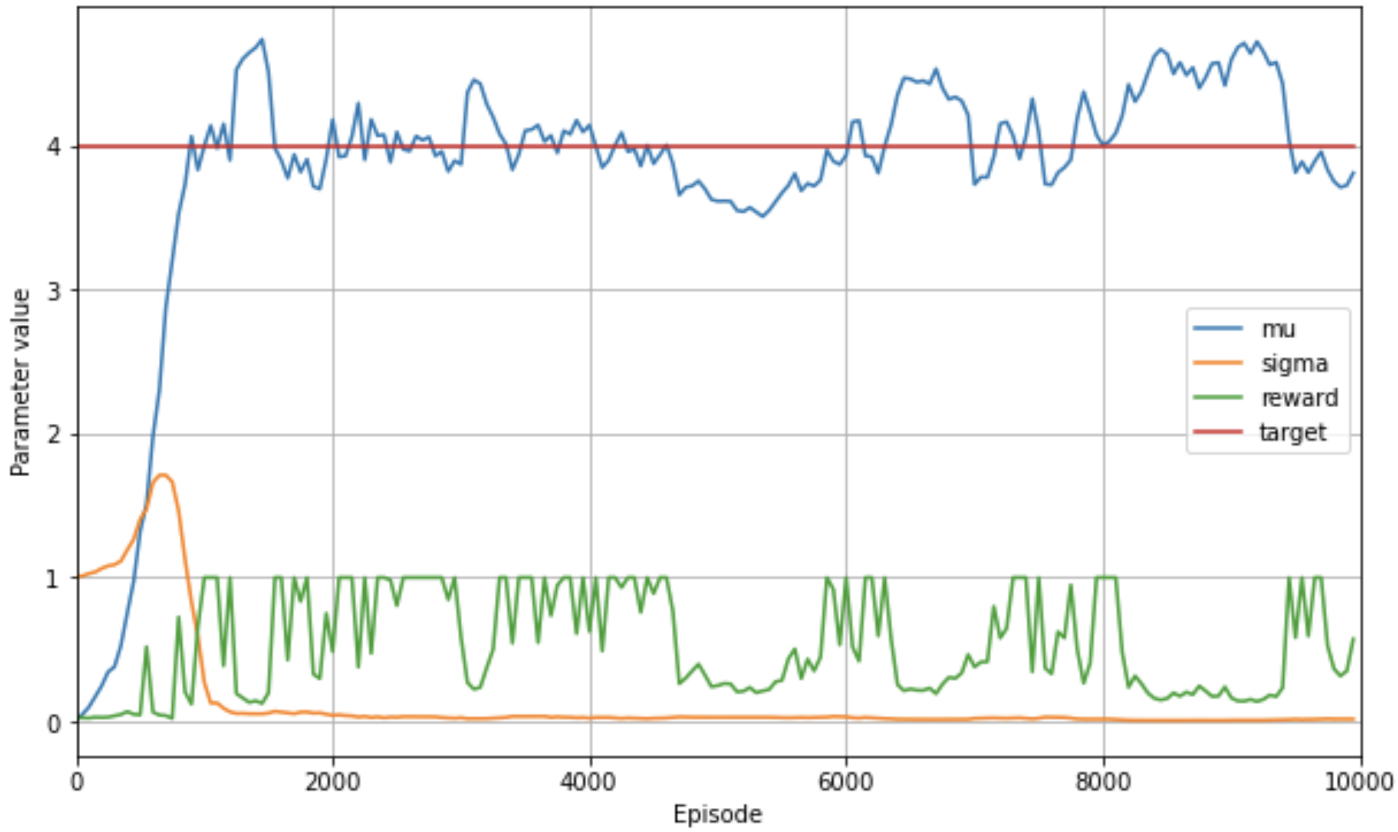


Fig from Schulman and Abbeel

Simple example

Simple example



Let’s go to the notebook

Summary so far

- Policy methods estimate $\pi(a \mid s)$
- Tabular methods can be used for small state/action spaces
- Otherwise, parameterize $\pi_\theta(a \mid s)$ and use gradient ascent
  - Change in expected reward calculated with “grad-log trick”

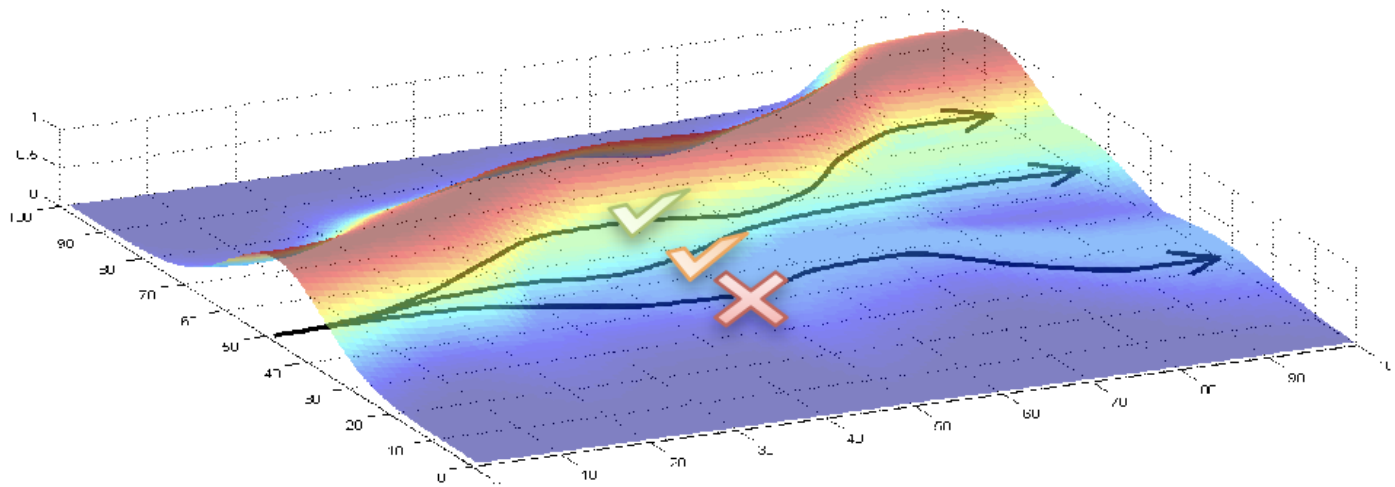
Next: Some improvements

- Baseline substraction
- Value function estimation
- Advantage estimation

Recall: Intuition

Gradient ascent of the expected reward with respect to policy:

- Increases probability of paths with large R
- Decreases probability of paths with small R



How to make this more efficient?

- If rewards are all positive, we might increase weight on all actions — with larger increases when rewards are greater
- We actually want to boost up actions associated with rewards that have greater than average reward
- Similarly, downweight actions associated with rewards that have below average reward
- This can *reduce the variance* of policy estimates

Baseline subtraction

Recall:

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) R(\tau^{(i)})$$

is an unbiased estimate of the gradient

Baseline subtraction

But then

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) \left(R(\tau^{(i)}) - b \right)$$

is *also* an unbiased estimate of the gradient, where b is a "baseline" average.

Why?

Baseline subtraction

Note that

$$\begin{aligned}
\mathbb{E}\left(\nabla_\theta \log p(\tau \mid \theta)\, b\right) &= \sum_\tau p(\tau \mid \theta) \nabla_\theta \log p(\tau \mid \theta)\, b \\
&= \sum_\tau \nabla_\theta p(\tau \mid \theta)\, b \\
&= \nabla_\theta \sum_\tau p(\tau \mid \theta)\, b \\
&= b \nabla_\theta \sum_\tau p(\tau \mid \theta) \\
&= b \nabla_\theta 1 \\
&= 0
\end{aligned}$$

Same result will hold as long as b doesn't depend on the actions a_t

Introducing time dependence

Now we can refine this. The gradient is

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta)(R(\tau^{(i)}) - b)$$

$$= \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \right) \left(\sum_{t=0}^{T} r_t^{(i)} - b \right)$$

$$= \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=0}^{t} r_k^{(i)} + \sum_{k=t+1}^{T} r_k^{(i)} - b \right) \right)$$

We then remove terms in the reward that don't depend on the current action $a_t^{(i)}$:

Here we're using the reward at step k as $r_k = \text{reward}(s_k, a_k)$

Introducing time dependence

We then replace this with

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b \right)$$

or

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b(s_t^{(i)}) \right)$$

We weight the gradients at time t by the total reward from that time onwards minus the average reward

This makes more efficient use of the samples.

Using the value function

But the average reward starting from the state s_t is the *value function*:

$$b(s_t) = \mathbb{E}(r_t + r_{t+1} + \cdots + r_T) = V^{\pi}(s_t)$$

So, we are led to simultaneously look for an estimate of the value function while estimating the policy.

This is a type of *actor-critic* reinforcement learning.

Estimating the value function

We use the improved (lower variance) gradient estimate

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - V^\pi(s_t^{(i)}) \right)$$

Estimate using regression for another neural network with parameters ϕ:

$$\phi \leftarrow \arg\min_\phi \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \left(V_\phi^\pi(s_t^{(i)}) - \sum_{k=t+1}^{T} r_k^{(i)} \right)^2$$

Can also use the Bellman equation to estimate V_ϕ, as in Q-learning.

Using the Bellman equation

- Roll out policy, collecting paths $\tau^{(i)}$ and instances of (s, a, s_{next}, r)
- Update value function:

$$\phi \leftarrow \arg\min_{\phi} \sum_{(s,a,s_{next},r)} (r + \gamma V^{\pi}_{\phi_{old}}(s_{next}) - V^{\pi}_{\phi}(s))^2 + \lambda \|\phi - \phi_{old}\|^2$$

- Update policy:

$$\theta \leftarrow \theta + \eta \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_{\theta} \log \pi(a_t^{(i)} \mid s_t^{(i)}) \underbrace{\left(\sum_{k=t+1}^{T} \gamma^{k-t-1} r_k^{(i)} - V^{\pi}_{\phi}(s_t^{(i)}) \right)}_{\textit{Advantage}}$$

Summary

- Policy gradients have a nice form
- Can reduce variance in gradient ascent by comparing to baseline
- Natural baseline is given by the value function
- Value function can be estimated during policy estimation
- Modern RL methods develop different refinements to this approach